

Ch09 - 웹 크롤러 설계 (발표)

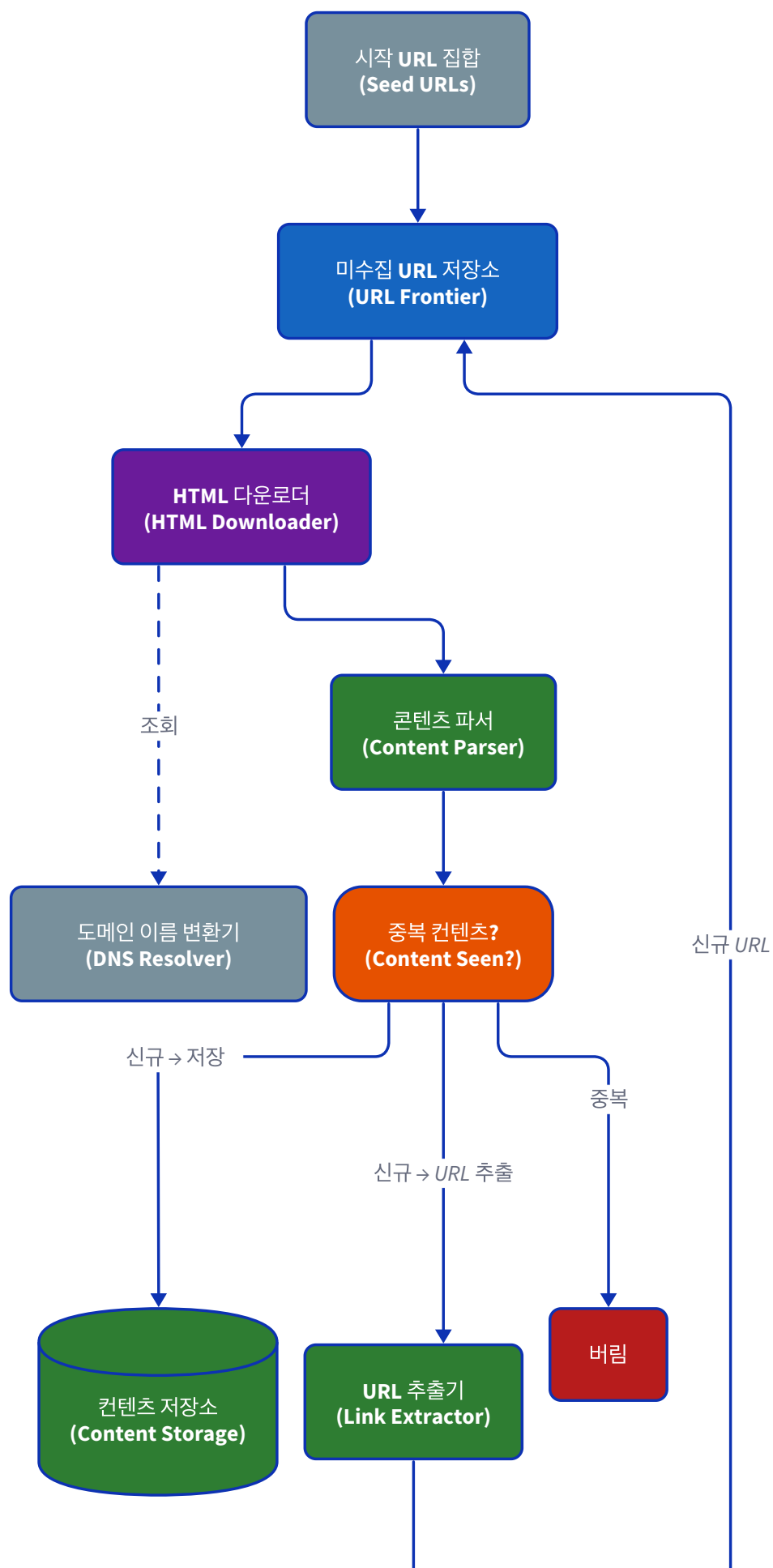
9장. 웹 크롤러 설계 (Design a Web Crawler)

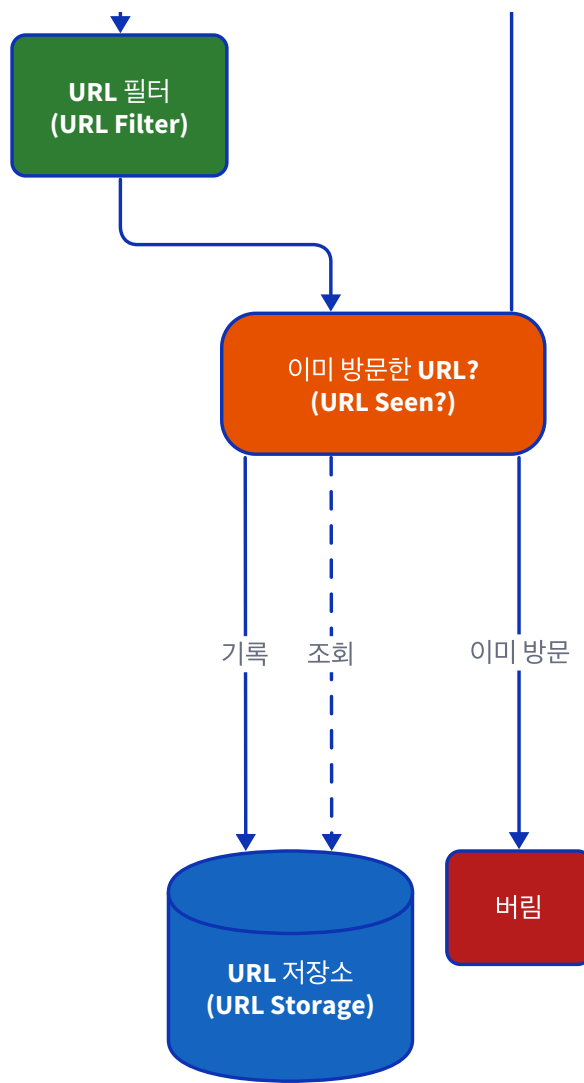
- 웹 크롤러의 종류
 - 아키텍처 설계
 - 규모 추정
 - 시작 URL 정하기
 - 수집 안한 URL 저장소
 - HTML 다운로더
 - 도메인 이름 -> IP 주소 변환기
 - 콘텐츠 파서
 - 중복 콘텐츠 비교
 - 콘텐츠 저장소
 - 다음에 크롤링할 URL 추출기
 - URL 필터
 - URL 저장소
 - 상세 설계
 - DFS vs BFS
 - DDOS 걸지 않기
 - 검색 우선순위 설정하기
 - 재수집 필요한 URL 정하기
 - 미수집 URL에 대한 지속성 저장장치
 - HTML 다운로더
 - Robots.txt
 - 성능 최적화 기법
 - 문제 있는 콘텐츠 감지/회피
-

Part 1.

1. 개략적 설계

전체 아키텍처





컴포넌트별 역할

한국어 명칭	영문	역할
시작 URL 집합	Seed URLs	크롤링 출발점. 설계자가 주제별/지역별로 선정
미수집 URL 저장소	URL Frontier	미방문 URL을 저장하고 우선순위 + Politeness 를 관리하는 큐
HTML 다운로드	HTML Downloader	Frontier에서 URL을 꺼내 웹 페이지를 다운로드
도메인 이름 변환기	DNS Resolver	URL → IP 변환. 10~200ms 병목 → DNS 캐싱으로 해결
콘텐츠 파서	Content Parser	다운로드한 HTML을 파싱/검증. 별도 컴포넌트로 분리
중복 콘텐츠?	Content Seen?	콘텐츠 해시 비교 로 중복 탐지 (웹페이지 29%가 중복)
URL 추출기	Link Extractor	HTML에서 링크 추출, 상대→절대 경로 변환
URL 필터	URL Filter	블랙리스트, 특정 확장자 등 제외
이미 방문한 URL ?	URL Seen?	블룸 필터 로 이미 방문한 URL인지 확인
URL 저장소	URL Storage	방문 완료 URL을 보관. URL Seen?이 이것을 조회

2. 상세 설계

2-1. URL 프론티어 : 미사용 URL의 순서 정하기

✍ 핵심 아이디어

- **DFS**: 한 경로를 끝까지 따라감 → 깊이 예측 불가 → 부적합
- **BFS**: 일반적으로 사용하지만, 순수 BFS의 한계를 **URL 프론티어**가 해결

BFS 문제

같은 호스트에 요청 폭주 → DDoS

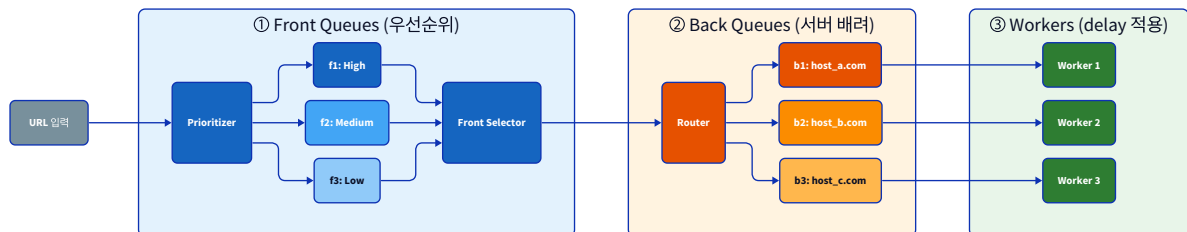
모든 URL을 동등 취급

URL 프론티어 해결책

Back Queues: 호스트별 큐 분리 + delay

Front Queues: 우선순위별 큐

URL 프론티어 = 2단계 구조



- ① **Front Queues** — Prioritizer가 PageRank, 트래픽, 갱신 빈도 등으로 URL 중요도를 판단하여 우선순위별 큐에 분배. 높은 우선순위 큐에서 더 자주 URL을 꺼냄
- ② **Back Queues** — 같은 호스트의 URL은 같은 큐에 넣고, **호스트당 하나의 워커만 접근**. Router가 호스트 → 큐 매핑을 관리
- ③ **Workers** — 각 워커는 자기 큐에서만 URL을 꺼내 다운로드. 워커 간 **delay**를 두어 서버에 부담을 주지 않음

② 고민: "중요한 페이지를 더 자주 재크롤링" — 근데 **중요하다는 걸 어떻게 판단**하지?

→ PageRank, QDF, Crawl Budget 등

2-2. 재수집 필요한 URL 정하기 (Freshness)

- 웹 페이지는 계속 변경됨 → **변경 이력** 기반 재크롤링 주기 결정
- HTTP **Last-Modified / ETag** 헤더로 조건부 GET → 변경 없으면 다운로드 생략
- 중요한 페이지를 더 자주 재크롤링, 모든 URL을 재크롤링하는 것은 비효율 → **선별적 재크롤링**

미수집 URL에 대한 지속성 저장장치

- 프론티어의 대부분의 URL은 디스크에 저장, 자주 접근하는 URL만 메모리 버퍼에 유지
- 주기적으로 버퍼 → 디스크 기록 (flush)

2-3. HTML 다운로더

Robots.txt

웹사이트가 크롤러에게 허용/차단 규칙을 명시하는 파일. 크롤링 전 반드시 확인, 결과를 **캐싱**하여 반복 다운로드 방지.

```
User-agent: Googlebot
Disallow: /creatorhub/*
Allow: /creatorhub/c/

User-agent: *
Disallow: /search
```

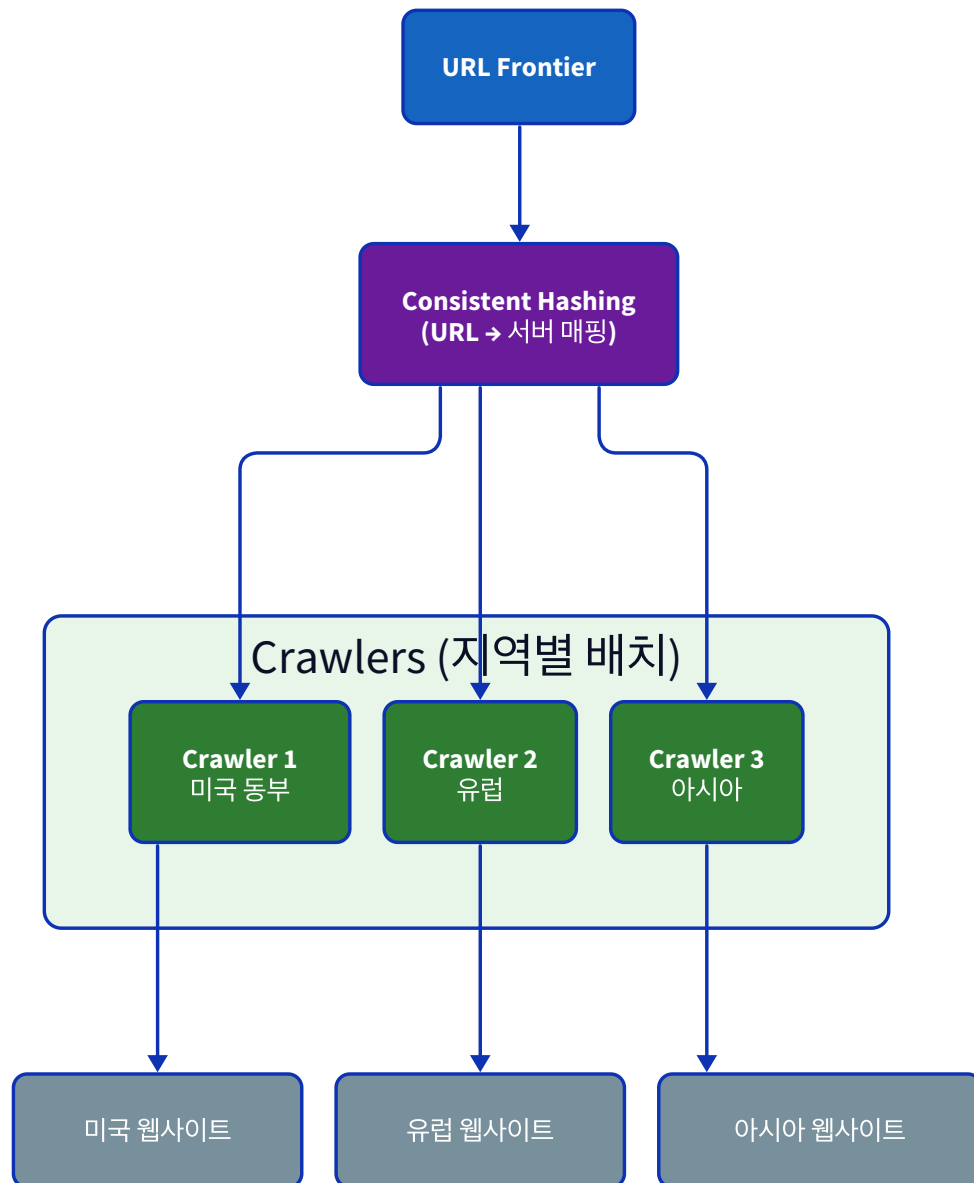
성능 최적화 기법

기법	효과
분산 크롤링	URL 공간을 분할하여 여러 서버가 담당
DNS 캐싱	dns 조회 결과 캐싱 조회할 때 생기는 10~200ms 병목 해소. 결과를 캐싱하고 주기적 갱신
지역성 (Locality)	크롤링 서버를 대상 웹사이트와 지리적으로 가까운 곳에 배치
짧은 타임아웃	응답 없는 서버에 시간 낭비 방지

분산 크롤링 아키텍처

 **안정 해시(Consistent Hashing)**

해시 링 위에 서버 노드를 배치해서, 노드 추가/제거 시 URL 재배포를 최소화하는 기법 (Ch05 - 안정 해시 설계 참고)



② 고민: 분산 환경에서 여러 크롤러가 동시에 크롤링하면 중복이 생기지 않을까?

문제 있는 콘텐츠 감지/회피

문제 유형	대응 방법
중복 콘텐츠 (웹의 ~30%)	해시/체크섬으로 탐지
스파이더 트랩 (/a/a/a/...)	URL 길이 제한 + 수동 블랙리스트
데이터 노이즈 (광고, 스크립트, 스팸)	필터링 로직 구현

🤔 고민: 책에서는 스파이더 트랩만 언급했는데, 실제로는 더 다양한 트랩이 있지 않을까?

→ 달력 트랩, 패킷 내비게이션, AI 미로 등

2-4. 중복 탐지 (블룸 필터)

레이어	대상	방법	위치
URL 중복	같은 URL 재방문 방지	블룸 필터	링크 추출 후
콘텐츠 중복	다른 URL, 같은 내용	해시 비교	다운로드 후

🔗 블룸 필터(Bloom Filter)란?

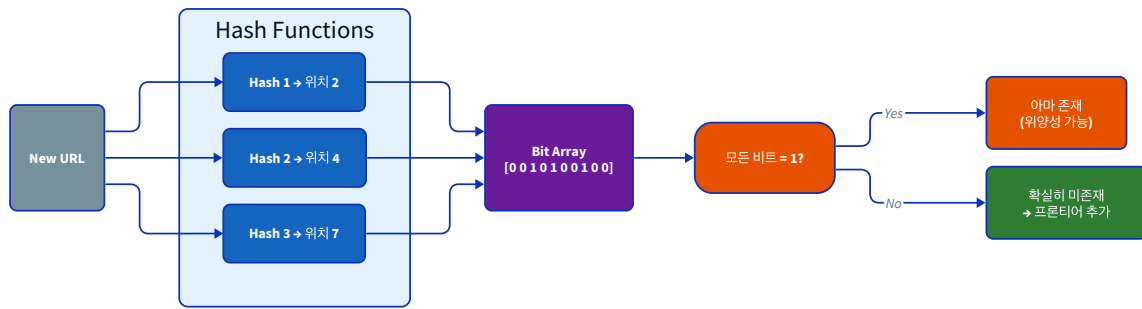
"이 URL을 이미 방문했는가?" 를 매우 적은 메모리로 빠르게 판단하는 확률적 자료구조.

10억 개 URL을 해시 테이블에 저장하면 수십 GB가 필요하지만, 블룸 필터는 약 **1.2GB**로 관리 가능.

대신 **100% 정확하지는 않다** — "있다"는 답이 틀릴 수 있지만(위양성), "없다"는 답은 항상 정확(위음성 없음).

동작 원리:

- 길이 m의 비트 배열을 0으로 초기화
- URL을 k개의 해시 함수에 넣으면 k개의 위치가 나옴
- 삽입: 해당 위치의 비트를 1로 설정
- 조회: k개 위치가 모두 1이면 "아마 존재", 하나라도 0이면 "확실히 없음"

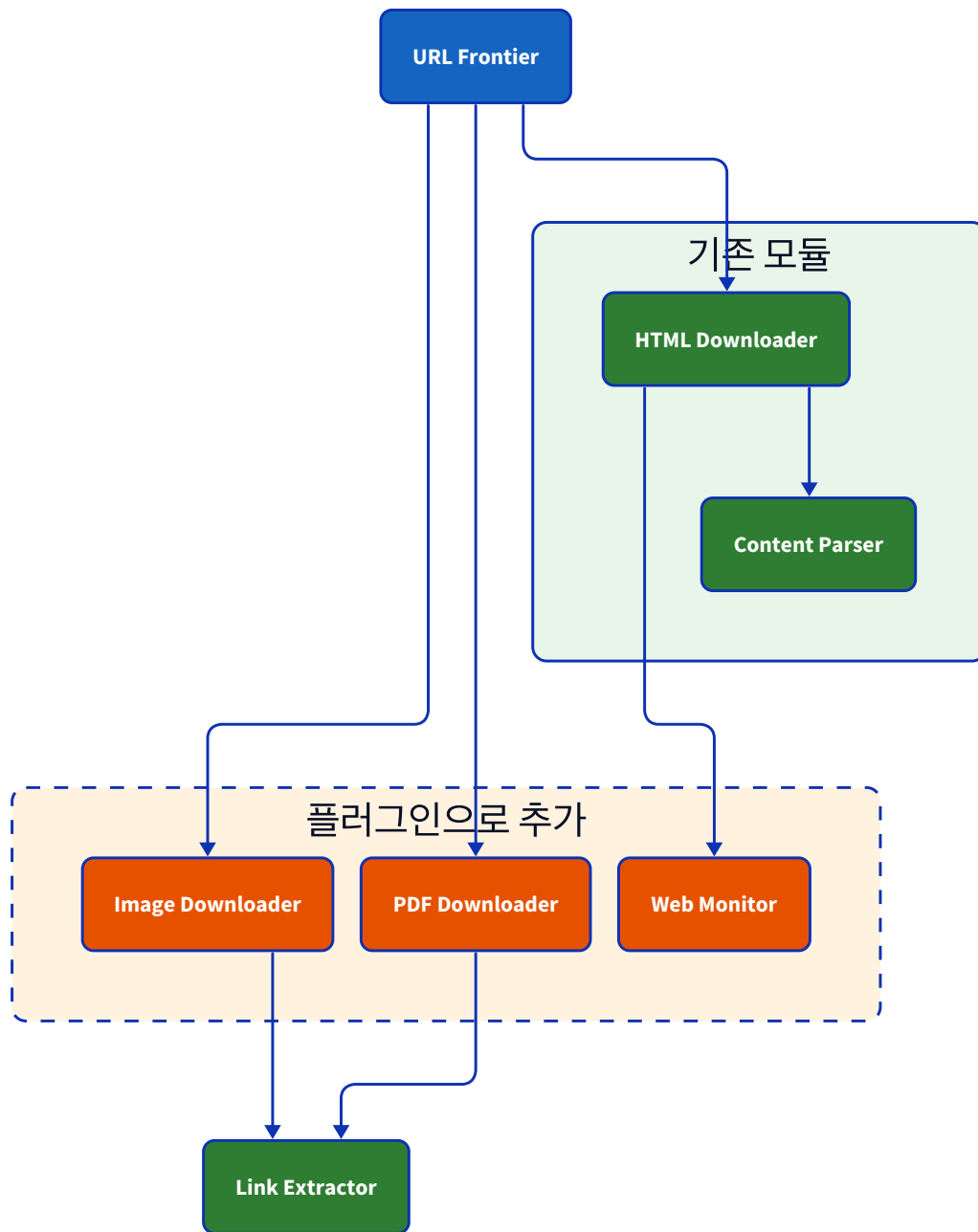


⚠ 왜 위양성(false positive)이 발생하는가?

서로 다른 URL이 우연히 같은 비트 위치에 해시될 수 있기 때문.

- 위양성 확률: 비트 배열 크기와 해시 함수 수로 조절 (1% 이하 가능)
- 위음성: 불가 — "없다"는 답은 항상 정확
- 공간 효율: 10억 URL을 약 **1.2GB**로 관리 (해시 테이블의 1/10)

2-5. 확장성: 모듈형 아키텍처



기존 코드 수정 없이 새 모듈을 끼워넣을 수 있는 구조

② 이미지/영상/PDF를 크롤링하면 어떻게 저장하지?

→ 메타데이터↔바이너리 분리, PB급 저장 기술

📖 4가지 속성 × 핵심 설계 결정

속성	핵심 설계 결정	기억할 것
Politeness (예의, 서버 배려)	URL 프론티어 (Front + Back Queues)	우선순위 + 호스트별 분리 + 신선도
Scalability (규모 확장)	안정 해시 + 블룸 필터	분산 크롤링, 위양성 trade-off
Robustness (안정성)	상태 저장 + 트랩 방어	체크포인트, 지수 백오프, 노이즈 필터링
Extensibility (기능 확장)	모듈형 아키텍처	기존 코드 무변경으로 새 모듈 추가

Part 2.

📖 목차

- 1. 실제 크롤링 규모는?
- 2. URL 중요도 판단 기준
- 3. 분산 환경에서의 중복 탐지
- 4. 크롤러 트랩 실제 사례
- 5. 멀티미디어 처리 + 대규모 저장 기술
- 6. AI 크롤러 시대의 robots.txt

1. 실제 크롤링 규모

책에서 제시한 크롤링 규모 : 월 10억 개의 URL
실제 사례는?

회사	월 크롤링 규모	출처
Common Crawl	25~30억 페이지	공식 통계
Meta	30억+ 페이지	Zuckerberg 실적발표 발언 (2024)
Internet Archive	~150억 페이지	공식 블로그 (2025.10)
Google	비공개 (누적 인덱스 ~4,000억 문서)	반독점 재판 법원 증언 (2020)

2. URL 중요도 판단 기준

PageRank

- 다른 페이지가 이 페이지를 많이 링크할수록 중요하다고 판단
- 원리: "중요한 페이지가 링크한 페이지도 중요하다"는 재귀적 계산
- 예: 수천 개 사이트가 링크하는 wikipedia.org → 높은 PageRank
- 한계: 링크 팜 (가짜 사이트 수천 개를 만들어 서로 링크) 에 취약

갱신 빈도

- 자주 바뀌는 페이지를 더 자주 방문하는 전략
- 뉴스 사이트 → 매시간 재크롤링 / 회사 소개 페이지 → 한 달에 한 번
- 과거 변경 이력을 보고 "이 페이지는 평균 3일마다 바뀌니까 3일마다 방문"

트래픽/인기도

- 실사용자가 실제로 많이 방문하는 페이지를 우선 크롤링
- Google은 Chrome 브라우저 사용 데이터, 검색 클릭 데이터 등을 활용
- 한계: 신규 페이지는 트래픽 데이터가 없어서 판단 불가

도메인 권위 (Domain Authority)

- 개별 페이지가 아니라 도메인 전체의 신뢰도를 평가
- nytimes.com → 높은 권위 / spam-blog-12345.com → 낮은 권위
- 스팸 도메인을 사전에 걸러내는 데 유용

QDF (Query Deserves Freshness)

- "이 주제가 지금 핫하다" → 관련 URL 우선순위를 동적으로 올림
- 예: 올림픽 기간에 "올림픽" 관련 검색량 급증 → 올림픽 관련 페이지 크롤링 우선순위 상향
- 뉴스 발행량, 소셜미디어 언급량 급증이 트리거

Google의 Crawl Budget

- Google이 한 사이트에 할당하는 크롤링 자원의 한도
- $\text{Crawl Budget} = \text{Crawl Capacity} \times \text{Crawl Demand}$
 - **Crawl Capacity**: 서버가 감당할 수 있는 크롤링 속도 (서버가 느려지면 자동 감속)

- **Crawl Demand:** Google이 이 사이트를 얼마나 크롤링하고 싶은가 (인기도, 갱신 빈도 등)
- 소규모 사이트는 거의 영향 없고, 수백만 URL 이상의 대규모 사이트에서만 실질적으로 중요

② 토론: 크롤링 초기에 PageRank vs 트래픽 기반 중 어느 쪽이 더 유효한가?

- **PageRank:** 링크만 있으면 계산 가능 → 초기에 쓸 수 있지만 조작에 취약
- 트래픽: 정확하지만 신규 페이지에는 데이터 없음
- 실제로는 여러 시그널을 가중치를 두어 복합적으로 사용

3. 분산 환경에서의 중복 탐지

🔗 여러 크롤러가 동시에 돌아가는데 중복이 안 생기는 이유

- **URL 중복:** 안정 해시로 같은 URL은 항상 같은 크롤러가 담당 → 구조적으로 방지
- **콘텐츠 중복:** 다른 URL이지만 같은 내용 (예: ?ref=abc) → 공유 저장소에서 해시 비교
- 단순 해시는 완전히 동일한 문서만 탐지. 근사 중복은 **SimHash** 필요

② 블룸 필터 위양성으로 일부 페이지를 영영 발견 못하는 건 수용 가능한가?

- 수용 가능한 이유: 메모리 효율 >> 약간의 누락
- 최신 대안: **Cuckoo Filter** (삭제 지원), **Xor Filter** (더 작고 빠름)

4. 크롤러 트랩 실제 사례

🔗 스파이더 트랩만 있는 게 아니다

트랩 유형	예시	위험도
무한 경로	/a/a/a/a/... 무한 깊이	높음
달력 트랩	/events?year=3026 — 수천 년 뒤까지 생성	높음
패킷 내비게이션	필터 40개 조합 = 2^40 (1조+) URL	매우 높음

트랩 유형	예시	위험도
세션 ID 트랩	/page?PHPSESSID=abc123 — 매번 새 URL	중간
허니팟 링크	display:none 숨겨진 링크 → 봇만 클릭 → IP 차단	중간
AI 미로	Cloudflare AI Labyrinth (2025) — AI 가짜 콘텐츠로 봇을 가둠	최신

책의 대응: "URL 길이 제한 + 수동 블랙리스트"

실무: URL 정규화 + 최대 depth 제한 + 패턴 자동 탐지 + 도메인별 페이지 수 제한 조합

☹️ 6,000페이지 사이트에서 달력 트랩으로 100만+ URL이 생성된 사례가 있다. 자동 탐지 규칙을 만든다면?

5. 멀티미디어 처리 + 대규모 저장 기술

콘텐츠 유형	처리 방식	저장 위치
이미지	, OG 태그에서 URL 추출 → 별도 fetch	원본: S3, 메타데이터: DB
영상	파일 자체는 저장 안 함. 메타데이터/썸네일만 수집	메타데이터: DB, 썸네일: S3
PDF/문서	Apache Tika (1000+ 포맷), 스캔본은 OCR	텍스트: DB, 원본: S3

- Google: 콘텐츠별 전용 크롤러 (Googlebot, Googlebot-Image, Googlebot-Video)
- 핵심 패턴: 메타데이터 ↔ 바이너리 분리

☰ 대규모 저장 기술 상세 >

용도	기술	사례
메타데이터	HBase, Cassandra, DynamoDB	빠른 조회/갱신
콘텐츠 원본	HDFS, S3, Colossus(Google)	대용량 벌크
아카이빙	WARC 포맷 (ISO 28500)	Common Crawl, Internet Archive

- Google: Bigtable 10EB+, 초당 70억 요청
- Common Crawl: WARC → AWS S3 무료 공개, 월 250~455 TiB
- Internet Archive: 자체 PetaBox (200PB, S3 대비 5~10배 저렴)
- 비용: S3 ~\$21K/PB/월, Glacier ~\$1K/PB/월

6. AI 크롤러 시대의 robots.txt

⚠ 크롤링은 비용, 돌아오는 이득은 없다

- ClaudeBot: 23,951 페이지 크롤링당 1건의 트래픽만 돌려줌
- GPTBot: 1,276 페이지당 1건
- 검색엔진은 트래픽을 보내줬지만, AI 크롤러는 학습만 하고 돌려주지 않음

- 새 표준 제안: llms.txt — AI 크롤러를 위한 별도 접근 규약
- 방어 기술: Cloudflare AI Labyrinth — AI 생성 가짜 콘텐츠로 봇을 가두는 방어 기법

7. 최신 트렌드 (2024-2026)

기술 변화

영역	책 (전통)	2024-2026 최신
URL 우선순위	PageRank + 우선순위 큐	RL 기반 적응형, CRAW4LLM (21% URL로 100% 성능)
Politeness	호스트별 큐 + 고정 delay	적응형 throttling + llms.txt
중복 탐지	블룸 필터 + 해시	Cuckoo/Xor Filter + SemHash (시맨틱 중복)
JS 렌더링	"추가 논의"로만 언급	Lightpanda (Chrome 대비 11배 빠름)
크롤러 패러다임	규칙 기반 HTML 파싱	에이전틱 크롤러 — AI가 페이지를 관찰하고 행동을 계획
AI 연동	없음	MCP 기반 크롤러 — AI 에이전트와 크롤러 직접 연동

규제/법적 동향 — 크롤링의 "무법지대"가 끝나고 있다

시기	동향
2025.07	Cloudflare, 모든 신규 도메인에서 AI 크롤러 기본 차단
2025.12	Google, SerpApi를 DMCA 위반으로 제소 (검색 결과 스크래핑)
2026.02	IAB, "AI Accountability for Publishers Act" 초안 발의 — 동의 없는 스크래핑에 연방 소송권
2026.08	EU AI Act 전면 시행 — 학습 데이터 출처 공개, 저작권 옵트아웃 존중 의무화

- GPTBot 차단 사이트: 560만 개 (1년 전 330만 대비 급증)

- AI 크롤러 차단 사이트 증가율: 336% YoY
- AI 저작권 소송: 70건+ 진행 중, "대규모 학습용 스크래핑이 fair use인가"는 아직 미확정

② 토론: EU AI Act 시행 후, 대규모 크롤링은 어떻게 변할까?

- 학습 데이터 출처 공개 의무 → 크롤링 데이터 추적 필요
- robots.txt가 "권고"에서 법적 구속력으로 바뀌는 추세